

PRD — Système multi-agents « idée → dossier » (100 % local & gratuit)

Type : Product Requirements Document / Cahier des charges
Version : 1.1 (MVP) · **Date :** juin 2026
Contrainte cardinale : tourne **entièrement en local**, **zéro coût récurrent** (le seul « coût » est l'électricité). Aucune dépendance payante, aucune clé API obligatoire. Outil **personnel** (mono-utilisateur).

1. Vision & objectifs

Un système d'agents qui, à partir d'une **idée** (une phrase ou un paragraphe), produit automatiquement un **dossier complet + un PRD** : skills & connecteurs Claude pertinents, analyse concurrentielle, cadre juridique, recommandations de stack et plan de build — comme les documents produits à la main, mais déclenchés à volonté et **sans rien payer**.

Objectifs

- **O1.** Idée brute → dossier exploitable en local, sans relancer les recherches à la main.
- **O2. 0 € de coût récurrent :** modèles locaux (Ollama) + outils open-source.
- **O3.** Sortie **réutilisable** : Markdown (prêt pour le repo / Claude Code) + PDF lisible.
- **O4. 100 % privé :** aucune donnée ne quitte la machine (hors requêtes de recherche web).

Principe directeur honnête

La gratuité totale impose le **local**. Les modèles locaux (7B–30B) sont bons mais **n'égalent pas** Claude/GPT : la recherche sera un peu moins fine et plus lente. C'est un compromis acceptable pour un outil perso. On compense par : des **citations obligatoires**, du **recoupement de sources**, et un **agent vérificateur**.

2. Contraintes & principes

Principe	Implication
100 % local	LLM via Ollama (localhost), recherche et fichiers en local.
0 € récurrent	Aucune API payante. Seul coût = électricité + matériel déjà possédé.
Mono-utilisateur	Pas d'auth, pas de multi-tenant, pas d'hébergement cloud.
Dépendant du matériel	Le modèle utilisable dépend de ta VRAM/RAM (voir §5.1).
Hors-ligne sauf recherche	Tout fonctionne offline, sauf les requêtes web (nécessaires à la veille).

3. Périmètre du MVP

Dans le périmètre (MVP)

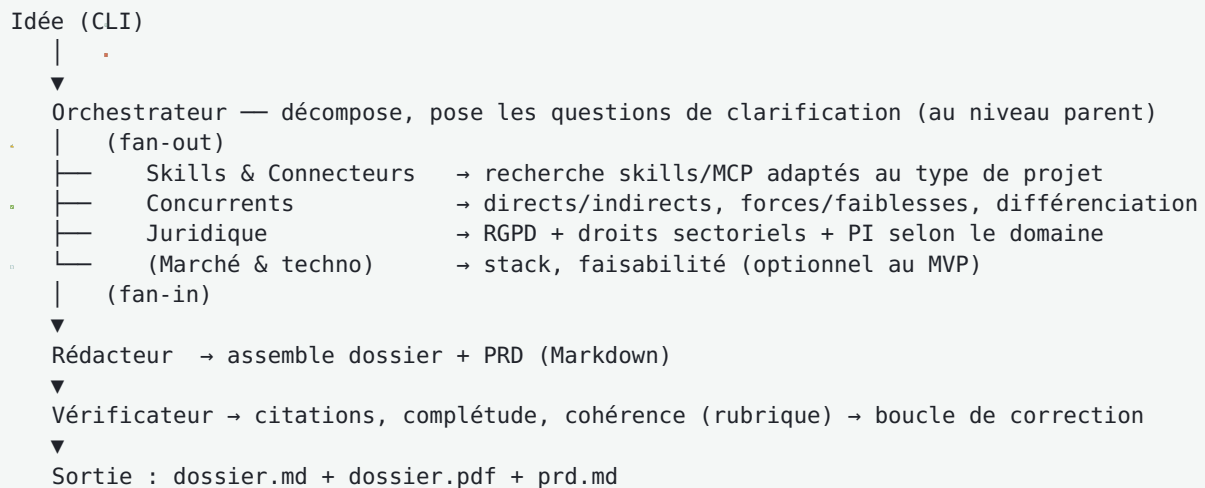
- Saisie d'une idée (CLI), génération d'un dossier + PRD en Markdown puis PDF.
- Orchestrateur + sous-agents : Skills&Connecteurs, Concurrents, Juridique, Rédacteur.
- Recherche web gratuite (DuckDuckGo sans clé / SearXNG auto-hébergé) + extraction de contenu.
- Citations des sources + agent vérificateur (rubrique).
- Sauvegarde locale des dossiers générés et d'un historique d'idées.

Hors périmètre (V2+)

- Hébergement cloud, multi-utilisateur, comptes.
- Modèles payants (laissés en option de repli gratuite seulement).
- UI web élaborée (le MVP est en **CLI** ; une UI locale simple est un *Should*).
- Veille planifiée/automatique récurrente.

4. Architecture (locale)

Motif **orchestrateur-ouvriers** : un orchestrateur décompose l'idée et délègue à des sous-agents, qui renvoient des **synthèses sourcées** ; le Rédacteur assemble, le Vérificateur contrôle.



Contrainte locale clé : la « fausse » parallélisation

Sur **un seul GPU local**, on ne peut pas vraiment faire tourner 4 instances de modèle en même temps (la VRAM ne suffit pas). Conséquences de conception :

- Au MVP, les sous-agents s'exécutent **en séquence** (ou avec une concurrence faible), pas en vrai parallèle comme dans le cloud.
- On choisit un **modèle compact et rapide** (tool-calling fiable) pour enchaîner les appels sans attente excessive.
- Repli optionnel : un **free tier cloud** (Groq/OpenRouter/Google AI Studio) pour de la vraie parallélisation, dans les limites gratuites.

Règle parent/sous-agent

Les questions de clarification et l'écriture des fichiers finaux restent gérées par l'orchestrateur (le parent), car un sous-agent ne peut ni poser de question ni demander d'autorisation en cours de route.

4.1 Ingénierie du contexte (principe transversal)

Le contexte est la ressource rare. Trois règles tirées des bonnes pratiques Claude Code, applicables aussi à notre système local :

- **Divulgaration progressive** : un skill au repos ne coûte que son nom + sa description (~100 tokens). Inutile de « désactiver » des skills pour économiser : le corps ne se charge qu'à l'invocation. Ce qui bloate vraiment, ce sont les **serveurs MCP** (définitions d'outils chargées en plein) → on en met **3-4 max**, branchés seulement quand utiles.
- **Fichier de règles court** : le `CLAUDE.md` (ou son équivalent système) reste **< 200 lignes** ; au-delà il grignote le contexte et l'adhérence baisse. Les détails vont dans des skills (chargés à la demande) ou la mémoire.
- **Résumés, pas transcripts** : chaque sous-agent renvoie une synthèse compacte, jamais son brouillon complet.

4.2 Activation par tâche : des sous-agents SCOPÉS (et non une désactivation manuelle)

« N'activer que ce qui sert pour cette tâche précise » ne se fait pas en cochant/décochant des skills à la main, mais en **scopant chaque sous-agent**. Chaque agent ne reçoit que les outils, skills et le modèle dont sa tâche a besoin :

Sous-agent	tools	skills injectés	model	Note
Orchestrateur	délègue seulement (mode Delegate)	write-spec	fort	retire Edit/Write/Bash pour forcer la délégation
Chercheur-skills	WebSearch, WebFetch, Read	skill « recherche »	léger/ rapide	contexte isolé
Chercheur-concurrents	WebSearch, WebFetch	skill « recherche »	léger/ rapide	<code>context: fork</code> (recherche lourde)
Chercheur-juridique	WebSearch, WebFetch	skill « recherche »	léger/ rapide	sources officielles
Rédacteur	Read, Write	docx, pdf	fort	assemble dossier + PRD
Vérificateur	Read, Grep (lecture seule)	—	léger	citations + complétude

Mécanismes Claude Code mobilisés : le champ **skills** du frontmatter d'un sous-agent injecte uniquement les skills voulus ; **disable-model-invocation: true** réserve un skill à l'appel explicite ; **context: fork** exécute un skill dans un sous-agent isolé. Résultat : aucun skill n'est « désactivé » globalement — chaque agent ne charge que le sien, et le reste profite de la divulgation progressive.

4.3 Mémoire auto-évolutive (le `.md` modifié automatiquement)

Pattern **capturer** → **distiller** → **réinjecter**, sur deux plans :

- **Côté Claude Code (pendant le dev)** : active l'auto-mémoire (Claude écrit lui-même ses apprentissages dans `MEMORY.md`) ; le raccourci `#` enregistre une note directement en mémoire projet ; des **hooks** `SessionEnd` / `PreCompact` capturent les décisions de la session et les écrivent dans `MEMORY.md`, qu'un hook `SessionStart` réinjecte au démarrage. Garder l'index < ~200 lignes et **élaguer** régulièrement (sinon le signal se noie).

- **Côté système (fonctionnalité produit)** : à chaque dossier généré, le système **écrit/append** un `MEMORY.md` local de mes préférences (stack favori, langue FR, format de livrable) et des **sources fiables** rencontrées, réutilisé au run suivant pour personnaliser et accélérer. Implémenté en Python (SQLite/JSON + `MEMORY.md`), pas via Claude Code.

Garde-fou : la mémoire est du **contexte**, pas une config imposée. Pour un blocage dur (ex. interdire d'écrire une clé en clair), utilise un **hook** `PreToolUse`, pas la mémoire.

5. Stack technique 100 % gratuite

5.1 Cœur LLM — Ollama (local, gratuit, illimité)

Ollama expose une API compatible OpenAI sur `http://localhost:11434/v1`. Choix du modèle **selon ta VRAM** (quantization `Q4_K_M` par défaut = ~95 % de la qualité, bon compromis) :

VRAM / RAM	Modèle conseillé (tool-calling)	Commande
8 Go	Qwen3 8B (ou Llama 3.1 8B)	<code>ollama pull qwen3:8b</code>
16 Go	Qwen3.5/3.6 ~27B (Q4) ou Gemma 4 26B MoE	<code>ollama pull qwen3.6:27b</code>
24 Go+	Qwen3 Coder 30B (long contexte)	<code>ollama pull qwen3-coder:30b</code>
CPU only	Qwen3 8B / Gemma 4 petit format	lent (~15-20 t/s), OK pour tester

La famille **Qwen** est la plus fiable pour le *tool-calling* (appel d'outils) en local, ce qui est critique pour un agent qui enchaîne recherches et écritures. Contexte ≥ 32K recommandé (`OLLAMA_CONTEXT_LENGTH`).

5.2 Recherche web — gratuite, sans clé

- **DuckDuckGo** (bibliothèque Python `ddgs / duckduckgo_search`) : recherche sans clé ni compte.
- ou **SearXNG** auto-hébergé en local (méta-moteur open-source, plus robuste, gratuit).
- **Extraction de contenu** : `httpx + trafilatura` (ou `readability`) pour récupérer le texte propre d'une page. Tout open-source.

5.3 Orchestration — open-source

- **Option recommandée MVP** : orchestrateur **maison en Python** (`asyncio` + client OpenAI pointant sur Ollama). Zéro dépendance lourde, contrôle total, gratuit.
- **Alternatives** : **CrewAI** ou **LangGraph** (open-source) si tu veux une structure multi-agents prête à l'emploi ; ou **Hermes Agent** (Nous Research, MIT) qui gère mémoire + skills et accepte n'importe quel backend compatible OpenAI (donc Ollama).

5.4 Génération des livrables — gratuite

- **Markdown** : généré directement par le Rédacteur.
- **PDF** : `markdown` → HTML → **WeasyPrint** (ou `pandoc`). Open-source.

5.5 Interface — locale

- **MVP : CLI** (`python run.py "mon idée"`).
- **Should : UI locale** via **Streamlit** ou **Gradio** (gratuit, tourne sur `localhost`), pour coller l'idée et voir/télécharger le dossier.

5.6 Persistance & mémoire — locale

- **SQLite** ou simples fichiers **JSON/Markdown** : historique des idées, cache des recherches. Aucun service externe.
- **MEMORY.md** **auto-évolutif** (voir §4.3) : préférences (stack favori, langue FR, format) + sources fiables, écrites/append à chaque run et réinjectées au suivant. Élagage automatique si > ~200 lignes.

5.7 Repli « free tier » (optionnel, non requis)

Si le local est trop lent/limité : **Groq** (~14 400 req/jour, 30 req/min, sans CB), **OpenRouter** (modèles gratuits avec tool-calling : Gemma 4 26B, Llama 4 Maverick), **Google AI Studio** (Gemini Flash, ~1 500 req/jour). *Limites de débit, susceptibles de changer — à n'utiliser qu'en complément.*

6. Parcours utilisateur (CLI)

1. `python run.py "une app de covoiturage pour festivals"` (ou via l'UI locale).
2. L'orchestrateur reformule l'idée et, si besoin, **pose 1-3 questions** de clarification dans le terminal.
3. Les sous-agents s'exécutent (barre de progression : Skills → Concurrents → Juridique → ...).
4. Le Rédacteur assemble `dossier.md` ; le Vérificateur contrôle les citations.
5. Conversion en `dossier.pdf` + génération de `prd.md`.
6. Les fichiers apparaissent dans `./out/<slug-idee>/` et l'idée est ajoutée à l'historique.

7. Fonctionnalités (epics) & priorisation MoSCoW

M Must · **S** Should · **C** Could · **W** Won't (MVP)

E1 — Entrée & orchestration

- **M** Saisie d'une idée en CLI ; reformulation + détection des décisions structurantes.
- **M** Questions de clarification au niveau parent (1-3 max).
- **M** Décomposition en sous-tâches + exécution séquentielle des sous-agents.
- **S** Concurrence limitée si la VRAM le permet.

E2 — Sous-agents de recherche

- **M** Skills & Connecteurs ; Concurrents ; Juridique (chacun = un *prompt/skill* avec sources & format de sortie).
- **M** Recherche web gratuite + extraction de contenu + **citations**.
- **S** Marché & techno (stack, faisabilité).
- **C** Cache des recherches pour réutilisation.

E3 — Rédaction & livrables

- **M** Rédacteur : assemble un dossier structuré (gabarit réutilisable) à partir des synthèses.
- **M** Génération Markdown + PDF.
- **M** Génération d'un PRD à partir du dossier.
- **S** Gabarits configurables (sections activables/désactivables).

E4 — Vérification qualité

- **M** Vérificateur à rubrique : présence des citations, complétude des sections, cohérence.
- **S** Boucle de correction automatique (renvoi au Rédacteur si la rubrique échoue).
- **C** Recoupement ≥ 2 sources imposé sur les faits sensibles.

E5 — Persistance & mémoire auto-évolutive

- **M** Sauvegarde locale des dossiers + historique des idées.
- **M** `MEMORY.md` auto-évolutif : à chaque run, écriture/append des préférences (stack, langue FR, format) et des sources fiables, réinjectées au run suivant (voir §4.3).
- **S** Élagage automatique du `MEMORY.md` (> ~200 lignes) pour éviter le bruit.

E6 — Interface

- **M** CLI.
- **S** UI locale (Streamlit/Gradio).

Reporté (W)

- Cloud, multi-utilisateur, veille planifiée, modèles payants par défaut.

8. Modèle de données (local)

Entité (table SQLite ou fichiers)	Champs clés	Rôle
ideas	id, text, slug, created_at, status	Historique des idées soumises.
runs	id, idea_id, model_used, duration_s, created_at	Une génération de dossier.
sections	id, run_id, agent, content_md, status	Sortie de chaque sous-agent.
sources	id, run_id, url, title, fetched_at, trusted (bool)	Sources citées (traçabilité) + marquage des sources fiables.
prefs	key, value	Préférences (stack, langue, format).
cache	query_hash, results_json, created_at	Cache des recherches web.
Fichiers	./out/<slug>/dossier.md , dossier.pdf , prd.md ; MEMORY.md (préférences + sources fiables, auto-évolutif)	Livrables + mémoire.

9. Exigences non-fonctionnelles

Domaine	Exigence
Coût	0 € récurrent. Aucune dépendance payante obligatoire.
Confidentialité	Tout en local ; seules les requêtes de recherche sortent. Aucune télémétrie.
Performance	Modèle adapté à la VRAM ; viser un dossier en quelques minutes (selon matériel).
Fiabilité	Citations obligatoires ; recoupement sur faits sensibles ; vérificateur.
Robustesse	Gérer les sources bloquées (ex. certains sites refusent l'accès auto) → sources alternatives ; timeouts et retries.
Portabilité	Windows (ton environnement) ; Python + Ollama ; pas d'OS-lock.
Maintenabilité	Chaque sous-agent isolé (prompt/skill) ; gabarits versionnés.

10. Qualité, limites & mitigations

Limite	Mitigation
Modèles locaux moins fins que Claude/GPT	Prompts précis + rubrique de vérification + recoupement ; repli free-tier ponctuel.
Pas de vraie parallélisation sur 1 GPU	Exécution séquentielle + modèle rapide ; option free-tier cloud pour paralléliser.
Hallucinations	Citations imposées ; refuser une affirmation non sourcée ; doc technique via recherche ciblée.
Sources web bloquées / bruitées	Méta-moteur (SearXNG), extraction propre (trafilatura), sources multiples.
Lenteur sur petit matériel	Modèle 8B + contexte réduit ; ou repli free-tier.
Le dossier juridique reste indicatif	Avertissement « non-conseil juridique » dans chaque sortie.

11. Roadmap / phasage

- **Phase 0 — Cadrage** : figer le gabarit du dossier + du PRD ; choisir le modèle Ollama selon ta VRAM.
- **Phase 1 — MVP** : CLI → orchestrateur + 3 sous-agents SCOPÉS (Skills&Connecteurs, Concurrents, Juridique) + Rédacteur + recherche DuckDuckGo + sortie MD/PDF.
- **Phase 2 — Qualité & mémoire** : Vérificateur + boucle de correction + recoupement + cache + `MEMORY.md` auto-évolutif (préférences + sources fiables).
- **Phase 3 — Confort** : sous-agent Marché&Techno, UI locale (Streamlit), gabarits configurables, enchaînement auto dossier → PRD, élagage mémoire.

12. Risques

Risque	Parade
Matériel insuffisant	Tier 8B + CPU fallback ; repli free-tier cloud.
Tool-calling cassé en local	Rester sur Qwen ; Ollama à jour ; contexte ≥ 32K.
Dérive des coûts (si on bascule cloud)	Garder le local par défaut ; plafonner les requêtes.
Qualité jugée insuffisante	Comparer 2 modèles sur tes idées ; affiner les prompts/rubriques.

13. Questions ouvertes (à trancher avant de coder)

- 1. **Matériel** : quelle est ta config (GPU + VRAM, RAM) ? → détermine le modèle Ollama exact.
- 2. **Orchestration** : orchestrateur maison Python (recommandé) ou framework (CrewAI/LangGraph/Hermes) ?
- 3. **Interface** : CLI seule au MVP, ou tout de suite une petite UI locale (Streamlit) ?
- 4. **Langue des dossiers** : français par défaut ?
- 5. **Profondeur** : on enchaîne automatiquement dossier → PRD, ou on s'arrête au dossier avec un PRD à la demande ?

14. Definition of Done (MVP)

- ☐ `python run.py "<idée>"` génère `dossier.md`, `dossier.pdf` et `prd.md` dans `./out/<slug>/`.
- ☐ Tout tourne en local via Ollama, **sans aucune clé API payante**.
- ☐ Au moins 3 sous-agents SCOPÉS (tools/skills/modèle propres à leur tâche) produisent des sections sourcées.
- ☐ Chaque affirmation importante est accompagnée d'une **citation** (URL).
- ☐ Le Vérificateur signale les sections incomplètes ou non sourcées.
- ☐ L'idée et ses livrables sont sauvegardés en local (historique).
- ☐ Un `MEMORY.md` local capture les préférences + sources fiables et est réinjecté au run suivant.
- ☐ Avertissement « non-conseil juridique » présent dans la sortie.
- ☐ Aucune donnée ne quitte la machine hormis les requêtes de recherche web.